

Music Knowledge Graph

A Semantic Web Application for Music Data Management

Course: Semantic Web 2025/26

Technologies: Python • Django • RDF • GraphDB • SPARQL • React

Dataset: 30000 Spotify Songs from Kaggle

Group 5
[112793] Cláudia Seabra
[129359] Nihal Fateen

April 14th, 2026

1. Introduction	3
1.1. Project Overview	3
1.2. Objectives	3
1.3. System Architecture	3
1.4. Technology Stack	4
2. Data, Sources, and Transformation	4
2.1. Dataset Description	4
2.2. RDF Transformation Process	4
2.3. Data Storage and Final RDF Graph Statistics.....	5
3. Operations on Data (SPARQL).....	6
3.1. Dual-Mode RDF Store Architecture	6
3.2. Query Patterns and Construction.....	6
3.3. Data Retrieval and Analytical Operations.....	6
3.4. Graph Modification and Maintenance	6
3.5. Hybrid Search	7
3.6. SPARQL Query Types Examples	7
4. Application Functionalities (UI)	8
4.1. System API Architecture.....	8
4.2 Frontend Features	8
4.3 Recommendation Engine Details	9
5. Conclusions	10
6. Configuration and Execution.....	11
6.1. System Requirements	11
6.2 Backend Setup	11
6.3. Verification.....	11
6.4. Troubleshooting.....	12

1. Introduction

1.1. Project Overview

The Music Knowledge Graph (MusicKG) is a comprehensive semantic web application designed to transform traditional relational music data into a queryable RDF-based knowledge graph. Unlike traditional Relational Database Management Systems (RDBMS), MusicKG treats musical entities, such as artists, albums, and tracks, as a network of interconnected nodes rather than isolated rows in a table.

This project uses Semantic Web technologies to create a flexible, interoperable, and semantically rich system environment for the management and exploration of musical discographies.

The knowledge graph approach provides several distinct advantages:

- Natural representation of complex relationships between artists, albums, tracks, and genres;
- Schema flexibility enabling evolution without breaking changes;
- Linked Data integration with external sources like DBpedia;
- SPARQL queries for complex graph traversal and pattern matching.

1.2. Objectives

The primary goal of MusicKG is to bridge the gap between raw data and semantic meaning. The specific objectives of this project are:

1. Transform a Kaggle Spotify playlist dataset into a semantically rich RDF knowledge graph;
2. Develop a custom music ontology with appropriate classes, properties, and relationships;
3. Implement a dual-mode RDF storage supporting GraphDB triplestore execution with a rdflib fallback for portability;
4. Build a comprehensive REST API to expose the knowledge graph through Django to external users;
5. Develop advanced features including similarity-based recommendations and timeline analytics;
6. Create an intuitive React-based frontend for real-time visualization of audio features and data exploration.

1.3. Semantic Approach and System Architecture

The application utilizes Semantic Web technologies to move beyond the initial structured tables. By implementing the Resource Description Framework (RDF), every entity (Artist, Album, and Track) is assigned a unique Uniform Resource Identifier (URI). This way the data is stored as [Subject, Predicate, Object] triples and by using a custom music ontology the data is ensured to adhere to defined semantic rules across the entire dataset.

MusicKG utilizes a three-tier architecture to ensure scalability and maintainability, beginning with the Triplestore Layer. As the foundation of the system, this layer uses GraphDB for high-performance RDF storage, handling both the persistence of triples and the execution of complex SPARQL 1.1 queries. This is supported by the Backend Logic Layer, where a Django-based API manages the core business logic, handles bulk data ingestion from CSV and NT sources, and maintains graph integrity through specialized update routines. Finally, the Frontend Presentation Layer employs a modern React interface to transform abstract graph data into interactive visualizations, such as audio feature distributions and similarity networks.

1.4. Technology Stack

Layer	Technology
Knowledge Graph	rdflib • RDF/NT • SPARQL 1.1 • Custom ontology (music:)
Backend	Python 3.9 • Django 4.2 • Django REST Framework
Triplestore	GraphDB Free Edition (with rdflib fallback)
Frontend	React 18 • Vite • Tailwind CSS • Recharts • Axios
Analytics Engine	NumPy • SciPy • Cosine similarity • Jaccard index

2. Data, Sources, and Transformation

2.1. Dataset Description

The project utilizes a comprehensive Spotify playlist dataset¹ sourced from Kaggle, containing detailed information about 32 833 music tracks. This raw data was originally obtained in CSV format, encompassing 23 distinct attributes that span basic metadata, audio features, and playlist categorization.

Dataset Statistics

Metric	Count
Total Tracks	32 833
Unique Artists	10 316
Unique Albums	20 728
Genres	6 (pop, rap, rock, latin, r&b, edm)
Attributes per Track	23

The source CSV contains the following key columns utilized in the transformation:

- Identifier: *track_id*
- Metadata: *track_name*, *track_artist*, *track_album_name*, *track_album_release_data*, *track_popularity*
- Audio Features: *energy*, *danceability*, *valence*, *tempo*, *loudness*, *loudness*
- Categorization: *playlist_genre*, *playlist_subgenre*

2.2. RDF Transformation Process

The transformation from CSV to RDF is implemented via a custom ETL (Extract, Transform, Load) engine built with Python, Pandas, and RDFLib.

Step 1: Ontology Definition

Prior to data loading, an OWL Ontology is initialized to establish the formal semantic rules of the Knowledge Graph. Defined under the <http://musickg.org/ontology#> namespace (prefix music:), this ontology distinguishes between Object Properties (linking entities) and Datatype Properties (linking entities to values). This ontology includes five main classes and their relationships, ensuring semantic consistency across the knowledge graph.

¹ <https://www.kaggle.com/datasets/joebeachcapital/30000-spotify-songs>

CSV Column	RDF Predicate	Object Type
track_name	music:trackName	xsd:string
track_artist	music:performedBy	URI (Artist)
track_album_name	music:albumName	xsd:string
energy	music:energy	xsd:float
playlist_genre	music:inGenre	URI (Genre)

Step 2: URI Generation

Each entity (artist, album, track) receives a unique URI following a consistent pattern, e.g., the group ‘Queen’ becomes <http://musickg.org/artist/Queen>. This hierarchical URI strategy prevents naming collisions and ensures global uniqueness. Artist URIs are generated using URL-safe slugs, while Album URIs are scoped to the artist using an MD5 hash of the Artist/Album pair. This allows the distinction between identical album titles produced by different artists. Track URIs are derived directly from the Spotify *track_id* to maintain a one-to-one mapping with the original source.

Step 3: Triple Generation

Each CSV row is converted into multiple RDF triples representing different aspects of the data. A single track generates approximately 15-20 triples covering metadata, relationships, and audio features.

Step 4: Named Graph Organization

Triples are organized into four named graphs for efficient querying and management: ontology definitions (101 triples), artist data (71 930 triples), album data (82 912 triples), and track data (414 765 triples).

- Ontology Graph (.../graph/ontology), which stores 101 triples defining OWL classes and constraints.
- Artist Graph (.../graph/artists) stores artist metadata and 10,322 owl:sameAs links to DBpedia.
- Album Graph (.../graph/albums) contains discography hierarchies and release years.
- Track Graph (.../graph/tracks) is the largest component and stores individual track facts and audio features.

Step 5: External Linking

Where possible, artists are linked to their DBpedia URIs using owl:sameAs predicates, creating 10 322 external links.

2.3. Data Storage and Final RDF Graph Statistics

Persistence is managed via a Dual-Mode Singleton Store (rdf_store.py) designed for high availability. GraphDB serves as the primary triplestore, handling SPARQL 1.1 operations via the RDF4J REST API. If the GraphDB server is unreachable, the system automatically falls back to an rdflib in-memory mode using a local N-Triples (.nt) dump, which maintains read-only functionality.

Component	Count
Total RDF Triples	569 708
Ontology Triples	101
Artist Triples	71 930
Album Triples	82 912
Track Triples	414 765
Similarity Links	30 948

Component	Count
DBpedia Links	10 322
File Size (NT format)	~75 MB

3. Operations on Data (SPARQL)

The interaction between the MusicKG application and the Knowledge Graph is managed through a dedicated logic layer that translates complex user actions into standardized SPARQL 1.1 queries. These operations ensure that all data remains semantically consistent and structurally sound across the triplestore.

3.1. Dual-Mode RDF Store Architecture

The system implements an innovative dual-mode architecture designed to maximize availability and reliability by seamlessly switching between a GraphDB triplestore and rdflib in-memory processing. GraphDB serves as the primary production-ready triplestore, mainly offering persistent storage and high-performance query execution typically ranging between 50ms and 300ms. On startup, the system pings the GraphDB REST API; if the server is reachable, it verifies repository existence and automatically handles data synchronization if the triple count is insufficient.

If the primary triplestore is unreachable, the system triggers a transparent fallback to rdflib, a pure Python RDF library. While this mode entails longer initial loading times (approximately 13 seconds) and slower query execution (up to 5 seconds), it guarantees that the application remains fully functional in a read-only state during maintenance or local development.

3.2. Query Patterns and Construction

All SPARQL operations utilize a standardized prefix block to ensure namespace consistency, including `music:` for the custom ontology and `base:` for instance data, alongside standard Semantic Web vocabularies like `rdf`, `rdfs`, and `xsd`. The application utilizes three primary SPARQL patterns: `SELECT` for complex data retrieval, `INSERT DATA` for entity creation, and `DELETE/INSERT WHERE` for state modifications.

3.3. Data Retrieval and Analytical Operations

Data retrieval is optimized through multi-stage operations to populate the user interface with interconnected metadata. Functions such as `get_artists` utilize `FILTER` and `LCASE` functions for case-insensitive matching and genre-pattern matching across the graph. For a detailed artist page, there is the `get_artist_detail` function. Instead of executing a one complex query that might struggle with large-scale aggregation, the system breaks the process into a series of smaller queries. This includes separate calls for metadata retrieval, genre aggregation, and discography mapping. This approach also utilizes `OPTIONAL` blocks to ensure that tracks lacking specific audio feature analysis are still included in results without breaking the query logic.

For high-level insights, e.g., the `get_genre_landscape` operation, SPARQL aggregation functions such as `COUNT` and `AVG` combined with `GROUP BY` clauses are used. This allows the system to calculate the "Audio Profile" of entire genres and enables the frontend to plot longitudinal trends and comparative analytics across the entire knowledge graph.

3.4. Graph Modification and Maintenance

The application also enables real-time modification of the Knowledge Graph. The `create_artist_node` function allows the user to add a new artist to the data. It initializes new entities by generating slug-based

URIs and performing pre-emptive duplicate checks. For discography management, *create_songs_bulk* and *update_track_album* manage the movement of tracks between albums. It is also possible to edit an album's release date via *update_album_year*. This function prevents data corruption by implementing a strict **DELETE/INSERT** pattern, so to guarantee that a node cannot accidentally accumulate multiple conflicting release year values.

A critical component of graph maintenance is the automated cleanup routine triggered by track-level modifications. Functions like *update_track_album* and *delete_track_from_graph* not only modify the relevant music:hasTrack relationships but also invoke a secondary "janitor" query. This routine identifies and deletes any parent album nodes that have been left with zero tracks so that there are no album nodes cluttering the graph and that the UI only displays valid and populated discographies.

3.5. Hybrid Search

To balance real-time accuracy with extreme performance, the system utilizes a **Hybrid Search Strategy** that executes a live SPARQL **UNION** query across artists, albums, and tracks to capture the most recent graph updates. These results are then merged with a 65 383-entry in-memory CSV index, resulting in sub-50ms full-text search capabilities without the overhead of a full SPARQL execution.

3.6. SPARQL Query Types Examples

SELECT Queries

Used for data retrieval. Example: retrieving all artists in the pop genre with their track counts:

```
PREFIX music:
<http://musickg.org/ontology#>
SELECT ?artist ?name (COUNT(?track)
AS ?count) WHERE {
  ?track music:inGenre
<http://musickg.org/genre/pop> ;
  music:performedBy ?artist .
  ?artist music:artistName ?name .
} GROUP BY ?artist ?name ORDER
BY DESC(?count)
```

FILTER Queries

Conditional filtering for refined results. Example: finding high-energy danceable tracks:

```
SELECT ?name ?energy ?dance
WHERE {
  ?track music:trackName ?name ;
  music:hasAudioFeatures ?af .
  ?af music:energy ?energy ;
  music:danceability ?dance .
  FILTER(?energy > 0.8 && ?dance >
0.7)
}
```

INSERT/DELETE Operations

SPARQL UPDATE operations enable dynamic graph modification. Example: adding a new artist:

```
INSERT DATA {
  <http://musickg.org/artist/New_Artist>
  a music:Artist ;
  music:artistName "New Artist" ;
  music:popularity 75 .
}
```

4. Application Functionalities (UI)

4.1. System API Architecture

The MusicKG interface is designed as a modern Single Page Application (SPA) using React 18.

Communication between the user interface and the Knowledge Graph is facilitated by the Django REST Framework (DRF), which powers 13 specialized API endpoints.

Category	Endpoint	Method	Functionality
Discovery	/api/artists/	GET	Paginated artist list with genre and search filtering.
	/api/artists/<slug>/	GET	Comprehensive artist profile including discography and analysis.
	/api/albums/<slug>/	GET	Album-specific metadata and full track listings.
	/api/tracks/	GET	Track search with deep audio feature range filters.
	/api/search/	GET	Full-text search across artists, albums, and tracks.
Analytics	/api/stats/	GET	Aggregated graph metrics and node/triple counts.
	/api/timeline/	GET	Global music trends and annual statistical distributions.
	/api/timeline/<genre>/	GET	Longitudinal evolution of audio features for a specific genre.
	/api/genre-landscape/	GET	High-level coordinate data for scatter plot visualizations.
	/api/audio-distribution/	GET	Histogram data for energy, danceability, and valence.
Graph Logic	/api/recommendations/<slug>/	GET	Similarity-based artist and track suggestions.
	/api/similar-edges/	GET	Similarity link data for the interactive Force Graph.
Modification	/api/artists/create/	POST	Initialization of new Artist nodes with URI generation.
	/api/songs/bulk-create/	POST	Batch ingestion of track and album nodes.
	/api/tracks/update-album/	POST	Re-assigning track nodes between album parents.
	/api/albums/update-year/	POST	Atomic update of album release year properties.
	/api/tracks/delete/	POST	Surgical removal of track nodes and orphan album cleanup.

4.2. Frontend Features

Artist Discovery & Profiles

The search engine serves as the primary entry point for graph exploration, featuring a dynamic sidebar for entity-type filtering and genre selection. The interface differentiates between Artists, Albums, and Tracks, so users can narrow the results according to what they are looking for.

Artist Analysis and Semantic Navigation

The Artist Detail page provides a comprehensive view of an artist's presence within the Knowledge Graph. The header displays real-time statistics derived from the graph, such as total album counts and track totals. Genres are rendered as interactive tags that link to filtered search results, facilitating semantic navigation across the graph. Furthermore, there is a connection to an external link to DBpedia resources, so the user can learn more about the artist.

A core feature of the application present in the artist's page is the visualization of the music:AudioFeatures nodes associated with each track. Average artist characteristics are mapped onto a Radar Chart, resulting in a visual representation of the artist's sonic identity across dimensions like Energy, Danceability, and Valence. This is complemented by detailed progress bars for specific metrics like Tempo and Loudness.

The discography is presented through an expandable album architecture, where AlbumCard components dynamically display tracks associated with specific album URIs.

Users can see the entire track list by the artist and sort it by popularity or audio characteristics. To facilitate graph maintenance, the UI includes an inline editing suite where users can change the tracks' album, triggering the SPARQL move-and-cleanup logic which ensures the discography remains structurally accurate. It is also possible to delete the track entirely, which is handled with a "Confirmation First" policy. If the user want to add missing songs they can by clicking on "Add tracks" and filling out a form with the song and respective album.

In the same page there are also recommendations. The engine combines two algorithms: 60% weight on cosine similarity of 5D audio feature vectors (energy, danceability, valence, normalized tempo, normalized loudness) and 40% weight on Jaccard similarity of genre sets. Results include similar artists with shared genres highlighted and 'You may also like' track suggestions from similar artists.

Album Page

Users can also open an album's page to see more details and correct the album's release date if necessary.

Timeline Analytics Page

The Timeline page lets users see how music trends have shifted over the years. A main chart shows how many songs were released each year alongside a line graph for a specific feature, like Energy or Danceability. Users can use sliders to pick a specific year range, from 1950 to 2024.

A "Highlights" sidebar quickly points out interesting facts, such as which year was the most energetic or when the most songs were produced. There is also a "Genre Evolution" section where users can pick a style, like "Rock" or "Pop" to see how its musical characteristics changed decade by decade.

Analytics Page

The Analytics page gives a big-picture view of the entire knowledge graph. A "Genre Landscape" scatter chart compares different genres by their energy and danceability. The size of the bubbles shows how many tracks each genre has, making it easy to see which styles dominate the library.

Users can also explore Audio Feature Distributions to see how features like Popularity or Tempo are spread across all songs. The page also includes a "Decade Comparison" chart, which shows how the average mood of music has evolved from the 1980s to the 2020s.

Graph Page

The Graph Page offers an interactive 2D map of the music world. It shows artists and genres as connected dots in a live network. Genres appear as large circles, and artists are smaller dots connected to their styles. Users can also see "Similarity edges" that link related artists together.

The graph is highly interactive: users can zoom in, search for a specific artist, or click a dot to see a summary of that node. To keep the map manageable, users can use a button to add 500 artists at a time.

4.3. Recommendation Engine Details

The similarity engine pre-computes a $10,316 \times 10,316$ artist similarity matrix at startup (approximately 9 seconds). This one-time computation enables $O(1)$ recommendation lookups at request time.

Algorithm 1: Audio Feature Cosine Similarity (60% weight)

For each artist, average audio features across all tracks are computed to create a 5-dimensional vector. Cosine similarity between normalized vectors provides a measure of audio characteristic similarity ranging from 0 (orthogonal) to 1 (identical).

Algorithm 2: Genre Jaccard Similarity (40% weight)

Genre sets for each artist are extracted from their tracks. Jaccard coefficient (intersection size / union size) measures genre overlap. For example, an artist with genres {pop, dance} has a coefficient of 0.33 with an artist having {pop, electronic}.

Combined Scoring

Final similarity score = $0.6 \times \text{cosine_similarity} + 0.4 \times \text{jaccard_similarity}$. Top 10 similar artists are returned sorted by combined score, along with shared genres and recommended tracks from the most similar artists.

5. Conclusions

The development of MusicKG successfully demonstrated the technical and practical advantages of utilizing Semantic Web technologies over traditional relational models for managing highly interconnected musical data. The success on transforming 32 833 Spotify tracks into a semantically rich RDF knowledge graph comprising 569 708 triples proved that a graph-based paradigm is superior for representing this type of data.

The biggest accomplishments of this project include the design of a custom music ontology and a comprehensive REST API and the implementation of a dual-mode RDF storage with automatic GraphDB detection and rdflib fallback. Moreover, the integration of SPARQL 1.1. to enable the execution of complex analytical queries and logical inferences, such as the direct materialization of artist-to-track links, that would have been much more complicated in a standard RDBMS.

On the frontend, the application successfully translated abstract RDF triples into meaningful human insights mainly by mapping the `music:AudioFeatures` class to interactive Radar Charts and distribution histograms so the user could take immediate conclusions about the artist's music.

Despite these successes, there are still areas to improve. Currently, when a user adds a new artist or track, it starts as a blank slate without the popularity scores or audio analysis (like energy and danceability) that come with the original Spotify data. In the future, it would be important to fix this by connecting the app to live services that can analyze new songs as they are added. It would also make the project more complete if the user had more control over the genres available and could add more.

Looking forward, the application could improve to include complex entities such as record labels, recording studios, live venues, and tour dates. Beyond metadata, the system could leverage its materialized `music:similarTo` relationships to develop an AI-driven recommendation engine based on graph embeddings, allowing for discovery based on path proximity rather than simple text matching. Ultimately, these potential developments highlight the scalability of the semantic approach and demonstrate that MusicKG is a living knowledge framework capable of evolving alongside the global music landscape.

6. Configuration and Execution

6.1. System Requirements

- Python 3.9+
- Node.js 18+ with npm
- GraphDB Free/Standard installed and running on port 7200

6.2 Backend Setup

1 — Generate the RDF graph

```
pip install -r requirements.txt
python convert_to_rdf.py \
  --csv spotify_songs.csv \
  --data-dir music_kg_project/data/
```

This creates music_kg.nt (569 708 triples, ~75MB) and stats.json in the data directory.

2 — Start the Django backend

```
cd music_kg_project
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Creates SQLite database for Django models (query templates, search logs). The RDF data lives separately in GraphDB or rdflib.

Server status: Runs at <http://127.0.0.1:8000>

GraphDB Check: Connects to port 7200

3 — Start the React frontend

```
cd music-kg-frontend
npm install
npm run dev
```

6.3. Verification

After starting both servers, verify functionality:

1. Visit <http://localhost:3000> to access the React frontend
2. Check <http://127.0.0.1:8000/>

3. Verify backend mode in console logs (GraphDB or rdflib)
4. Test search functionality with artist names like 'Ed Sheeran' or 'Taylor Swift'
5. Explore artist profiles and verify recommendation engine works

6.4. Troubleshooting

Issue: GraphDB not detected

Verify GraphDB is running on port 7200. Check firewall settings. Review Django console for connection errors. System will automatically fall back to rdflib.

Issue: Missing dependencies

Ensure Python 3.9+ and Node.js 18+ are installed. Reinstall requirements with `pip install -r requirements.txt --upgrade`. Clear npm cache with `npm cache clean --force`.